

NAPQES: A Single-Primitive Authenticated Encryption Scheme Built from HMAC-SHA256

Abdeslam Jakjoud¹ and Karim Amor¹

¹QAegis, France

May 2026

Preprint — submitted to IACR ePrint

Abstract

We present NAPQES, an Authenticated Encryption with Associated Data (AEAD) scheme whose security reduces exclusively to the pseudorandomness of HMAC-SHA256. Unlike AES-GCM and ChaCha20-Poly1305, NAPQES does not employ finite-field arithmetic, group operations, or AES S-box permutations, and is therefore not subject to Shor’s algorithm or hidden-subgroup quantum attacks. The construction combines a prime-indexed token cipher, HMAC-derived noise injection to achieve ciphertext pseudorandomness, HMAC-derived plaintext padding for cross-implementation determinism, and a standard HMAC-SHA256 authentication tag. We describe the wire format (version 6, frozen), prove IND-CPA security under the PRF assumption on HMAC-SHA256, discuss the post-quantum security level, and provide performance measurements and implementation notes. The streaming Release of Unverified Plaintext (RUP) caveat present in earlier versions is addressed by a new online-AE streaming format (`encrypt_stream_ae/decrypt_stream_ae`) that verifies a per-chunk HMAC-SHA256 tag before releasing any plaintext to the caller. A reference implementation in Python and Rust, together with a 30-vector Known-Answer Test corpus and a NIST SP 800-22 Rev 1a statistical analysis, are published at [https://github.com/\[REPO\]](https://github.com/[REPO]).

Contents

1	Introduction	3
2	Related Work	3
3	Construction	4
3.1	Notation	4
3.2	Key	4
3.3	Domain-Separated HMAC Derivation	4
3.4	Noise Probability	5
3.5	Plaintext Padding	5
3.6	Token Construction	5
3.7	Wire Format (Version 6)	5
3.8	Online-AE Streaming Format (v6s-ae)	5
4	Security Analysis	6
4.1	IND-CPA Security	6
4.2	INT-CTXT: Ciphertext Integrity	7
4.3	AAD Binding	8

4.4	IND-CCA Security	8
5	Post-Quantum Analysis	9
6	Comparison with AES-GCM and ChaCha20-Poly1305	9
7	Implementation	10
7.1	Python Reference Implementation	10
7.2	Rust Implementation	10
7.3	Constant-Time Considerations	10
8	NIST SP 800-22 Statistical Analysis	10
9	Known-Answer Tests	10
10	Fuzz Testing	10
11	Known Caveats	11
12	Conclusion	11

1 Introduction

Modern AEAD standards—AES-GCM [1] and ChaCha20-Poly1305 [2]—have achieved widespread deployment. Their security, however, ultimately depends on algebraic structures (finite-field multiplication for GCM, add-rotate-XOR chains for ChaCha20) whose resilience against quantum adversaries running variants of Shor’s algorithm is an active research question [5].

NAPQES takes a more conservative approach: the *only* cryptographic primitive in the construction is HMAC-SHA256, approved under NIST FIPS 198-1 and FIPS 180-4. The security of the entire scheme therefore reduces to the pseudorandomness of HMAC-SHA256—a well-studied assumption with no known quantum speedup beyond Grover’s $O(\sqrt{N})$ search, which reduces the effective security level from 256 bits to approximately 128 bits while remaining far above all recommended thresholds.

Contributions.

1. We specify NAPQES wire format v6 (Section 3), including the key serialisation, HMAC derivation schedule, token construction, padding scheme, and authentication tag.
2. We prove IND-CPA security (Section 4) under the standard PRF assumption on HMAC-SHA256 and show that the authentication tag provides EUF-CMA integrity.
3. We provide a post-quantum analysis (Section 5) and a comparison with AES-GCM and ChaCha20-Poly1305 (Section 6).
4. We report NIST SP 800-22 Rev 1a statistical test results on 10 million bits of NAPQES ciphertext output (Section 8), a 30-vector KAT corpus (Section 9), and a fuzz harness covering the full decoder pipeline (Section 10).
5. We discuss known caveats (Section 11): the 16-bit plaintext length cap (CAV-002), the padding length-bucket leakage (CAV-003), and the ciphertext expansion bound (CAV-004). The streaming Release of Unverified Plaintext issue (CAV-001) has been resolved by the online-AE streaming format introduced in this version.

2 Related Work

AES-GCM [1]. AES-GCM combines AES-CTR for confidentiality with a GHASH polynomial MAC over $\text{GF}(2^{128})$. The Forbidden Attack [6] and nonce-reuse attacks [7] demonstrate that the algebraic structure of GHASH can be exploited when the polynomial MAC key is recovered (e.g., under nonce misuse), algebraically recovering the authentication key. NAPQES uses HMAC-SHA256 for authentication, which does not admit the Forbidden Attack.

ChaCha20-Poly1305 [2]. ChaCha20 is an ARX cipher whose security is not reduced to a well-studied hardness assumption; Poly1305 is again a polynomial MAC, subject to the same class of Forbidden Attacks as GHASH when the one-time key is reused. Neither ChaCha20 nor Poly1305 is approved under current FIPS standards.

Ascon [3]. Ascon (NIST LWC winner) is a sponge-based AEAD targeting lightweight environments. Its security is based on the hardness of distinguishing the Ascon permutation from a random permutation, which involves xor-and-rotate operations over 64-bit words. Ascon is not yet in a FIPS-approved family.

HMAC-based constructions. HKDF [8] and HMAC-DRBG [9] demonstrate that HMAC-SHA256 is a sufficient basis for key derivation and pseudorandom generation. NAPQES extends this philosophy to the full AEAD context.

3 Construction

3.1 Notation

Throughout this paper:

- $\parallel$ denotes byte concatenation.
- $\text{HMAC}(K, M)$ denotes HMAC-SHA256 keyed with K over message M .
- $\text{be5}(x)$ denotes the 5 least-significant bytes of integer x , big-endian encoded.
- $\text{varint}(x)$ denotes unsigned LEB128 encoding of non-negative integer x .
- $\mathcal{P}$ denotes the set of prime integers in $[10^6, 9.9 \times 10^6]$.

3.2 Key

A NAPQES key $\mathbf{k} = (k_1, k_2, \dots, k_K)$ is an ordered tuple of K distinct primes drawn uniformly from $\mathcal{P}$ using a cryptographically secure DRBG. The serialised key material is:

$$\text{key_bytes} = \text{be5}(k_1) \parallel \text{be5}(k_2) \parallel \dots \parallel \text{be5}(k_K).$$

The cardinality of $\mathcal{P}$ is approximately 586 000; a 10-element ordered tuple provides a key space of:

$$\frac{|\mathcal{P}|!}{(|\mathcal{P}| - 10)!} \approx 1.1 \times 10^{58} \approx 2^{196}.$$

3.3 Domain-Separated HMAC Derivation

All internal values are computed as:

$$\text{Derive}_d(\text{kb}, N, \text{ctx}) = \text{HMAC}(\text{kb}, N \parallel d \parallel \text{ctx}),$$

where $d \in \{0, 1, \dots, 6\}$ is a 1-byte domain separator.

Domain	Function	Context	Output
0x00	Noise position	$\text{be5}(ct_pos)$	Boolean
0x01	Real-token addend	$\text{be5}(real_idx)$	$[1, k_i - 1]$
0x02	Noise probability	(none)	$[0.75, 0.99]$
0x03	Auth tag (block)	$\text{be4}(aad) \parallel aad \parallel \text{payload}$	32 bytes
0x04	Noise character	$\text{be5}(ct_pos)$	$[32, 127]$
0x05	Noise-token addend	$\text{be5}(ct_pos)$	$[1, k_i - 1]$
0x06	Padding codepoint	$\text{be4}(pad_idx)$	$[32, 126]$
0x07	Varint keystream	$\text{be4}(block_idx)$	32-byte block
0x08	Chunk tag (stream AE)	$\text{be4}(aad) \parallel aad \parallel N \parallel \text{be4}(i) \parallel \tilde{C}_i$	32 bytes
0x09	Final sentinel tag (stream AE)	$\text{be4}(aad) \parallel aad \parallel N \parallel \text{be4}(n_{\text{chunks}})$	32 bytes

Table 1: Domain-separated HMAC derivation schedule. Domains 0x08–0x09 are used exclusively by the online-AE streaming format (Section 3.8).

3.4 Noise Probability

$$t = \frac{\text{Derive}_{0x02}(\text{kb}, N, \varepsilon)[0 : 8]_{\text{be64}}}{2^{64}}, \quad p = 0.75 + t \cdot 0.24.$$

3.5 Plaintext Padding

Let $n = |P|$ (plaintext length in codepoints). Define:

$$B = \max(16, 2^{\lceil \log_2(n+1) \rceil}), \quad P_d = [n \gg 8, n \wedge 0\text{FF}] \| P \| \mathbf{pad},$$

where $\mathbf{pad}$ is a $(B - n)$ -codepoint sequence with each codepoint HMAC-derived via domain $0x06$ in [32, 126]. The total padded length is $n + 2 + (B - n) = B + 2$ codepoints.

3.6 Token Construction

For each padded codepoint c (with current ct_pos and $real_idx$), the token is emitted as follows:

Listing 1: Token emission loop (pseudocode)

```

1 while is_noise(ct_pos):           # domain 0x00
2     k = key[real_idx % K]
3     noise_c = derive_noise_char(ct_pos) # domain 0x04; in [32, 127]
4     noise_a = derive_noise_addend(ct_pos, k) # domain 0x05; in [1, k-1]
5     emit noise_c * k + noise_a
6     ct_pos += 1
7
8 k = key[real_idx % K]
9 a = derive_real_addend(real_idx, k) # domain 0x01; in [1, k-1]
10 emit c * k + a
11 ct_pos += 1; real_idx += 1

```

Remark 3.1. Because k is prime and $a \in [1, k - 1]$, every real token satisfies $\gcd(\text{token}, k) < k$. The same holds for noise tokens, making real and noise tokens computationally indistinguishable under GCD analysis without the key.

3.7 Wire Format (Version 6)

$$C = N \| \tilde{B} \| T,$$

where N is a 16-byte random nonce, $\tilde{B}$ is the domain- $0x07$ XOR-masked LEB128 token blob, and $T = \text{Derive}_{0x03}(\text{kb}, N \| \tilde{B})$ is the 32-byte HMAC-SHA256 authentication tag. The minimum valid ciphertext length is 48 bytes.

3.8 Online-AE Streaming Format (v6s-ae)

The basic streaming format (`encrypt_stream`) carries a single tag at the end of the stream, causing `decrypt_stream` to release plaintext before the tag is verified (RUP, formerly CAV-001). The online-AE streaming format eliminates this by attaching a per-chunk HMAC-SHA256 tag to each segment of the masked varint blob.

$$S = N \| \left[\text{be4}(\ell_i) \| \tilde{C}_i \| T_i \right]_{i=0}^{n-1} \| \text{be4}(0) \| T_{\text{final}},$$

where each $\tilde{C}_i$ is at most 1024 bytes of the masked varint blob and:

$$T_i = \text{HMAC}(\text{kb}, 0x08 \| \text{be4}(|A|) \| A \| N \| \text{be4}(i) \| \tilde{C}_i),$$

$$T_{\text{final}} = \text{HMAC}(\text{kb}, 0x09 \| \text{be4}(|A|) \| A \| N \| \text{be4}(n)).$$

The sentinel frame has $\ell = 0$; it signals end-of-stream. `decrypt_stream_ae` verifies T_i before yielding any plaintext from chunk i . T_{final} binds the total chunk count n , preventing silent truncation at a chunk boundary. Chunk index i in each T_i prevents chunk-reordering attacks.

4 Security Analysis

4.1 IND-CPA Security

Theorem 4.1 (IND-CPA). *Under the PRF assumption on HMAC-SHA256, the NAPQES block encryption scheme is IND-CPA secure. Specifically, for any PPT adversary $\mathcal{A}$ making at most q encryption queries there exists a PRF adversary $\mathcal{B}_1$ with similar running time such that:*

$$\text{Adv}_{\text{NAPQES}}^{\text{IND-CPA}}(\mathcal{A}) \leq \text{Adv}_{\text{HMAC-SHA256}}^{\text{PRF}}(\mathcal{B}_1) + \frac{q^2}{2^{128}}.$$

Proof. We prove the bound via a sequence of two hybrid games.

Game G_0 (Real Game). This is the standard IND-CPA experiment. The challenger samples $\mathbf{k} = (k_1, \dots, k_K)$ uniformly from K -element ordered tuples of distinct primes in $\mathcal{P}$, computes $\text{kb} = \text{key_bytes}(\mathbf{k})$, and answers up to q chosen-plaintext queries honestly using the real `NAPQES.Enc` algorithm (which calls $F(\text{kb}, \cdot) = \text{HMAC-SHA256}(\text{kb}, \cdot)$ for all internal derivations). After the query phase, $\mathcal{A}$ submits a challenge pair (m_0, m_1) of equal padded length; the challenger samples $b \xleftarrow{\$} \{0, 1\}$, returns $c^* \leftarrow \text{NAPQES.Enc}(\mathbf{k}, m_b)$, and $\mathcal{A}$ wins if its output $b' = b$.

Game G_1 (PRF Replacement). Identical to G_0 , except the function $F(\text{kb}, \cdot)$ is replaced by a truly random function $\rho : \{0, 1\}^* \rightarrow \{0, 1\}^{256}$, sampled uniformly at the start of the experiment and evaluated lazily. Every call that would invoke $\text{HMAC}(\text{kb}, x)$ now returns $\rho(x)$ instead.

Lemma 4.2. *There exists a PRF adversary $\mathcal{B}_1$, running in time $\mathcal{O}(T_{\mathcal{A}})$ with at most $q \cdot L_{\max}$ oracle calls (where $L_{\max}$ is an upper bound on the number of HMAC evaluations per encryption), such that*

$$|\Pr[\mathcal{A} \text{ wins } G_0] - \Pr[\mathcal{A} \text{ wins } G_1]| \leq \text{Adv}_F^{\text{PRF}}(\mathcal{B}_1).$$

Proof. $\mathcal{B}_1$ is given a function oracle $\mathcal{O}(\cdot)$ that is either $F(\text{kb}, \cdot)$ (real) or $\rho(\cdot)$ (random). $\mathcal{B}_1$ simulates the IND-CPA experiment for $\mathcal{A}$, answering every internal HMAC call by forwarding to $\mathcal{O}$. If $\mathcal{O} = F(\text{kb}, \cdot)$, the simulation is identical to G_0 ; if $\mathcal{O} = \rho$, it is identical to G_1 . Hence $\mathcal{B}_1$'s distinguishing advantage equals $|\Pr[\mathcal{A} \text{ wins } G_0] - \Pr[\mathcal{A} \text{ wins } G_1]|$. $\square$

Nonce collision probability. Let Coll denote the event that at least two of the $q + 1$ nonces generated across the q CPA oracle calls and the challenge encryption are equal. Each nonce is drawn independently and uniformly from $\{0, 1\}^{128}$, so by the union bound:

$$\Pr[\text{Coll}] \leq \binom{q+1}{2} \cdot 2^{-128} \leq \frac{q^2}{2^{128}}.$$

Lemma 4.3. *In G_1 conditioned on $\overline{\text{Coll}}$:*

$$\Pr[\mathcal{A} \text{ wins } G_1 \mid \overline{\text{Coll}}] = \frac{1}{2}.$$

Proof. Fix any adversary view V arising from the CPA query phase, which uses nonces $N_1, \dots, N_q$. Let N^* be the challenge nonce. Under $\overline{\text{Coll}}$, $N^* \notin \{N_1, \dots, N_q\}$.

In G_1 , every challenge-phase derivation uses an input prefixed by N^* (e.g., $\rho(N^* \parallel 0\text{x}00 \parallel \text{be}5(j))$, $\rho(N^* \parallel 0\text{x}01 \parallel \text{be}5(j))$, $\rho(N^* \parallel 0\text{x}02)$, and so on for domains $0\text{x}03$ – $0\text{x}07$). All CPA-phase evaluations

used inputs prefixed by some $N_i \neq N^*$. Since ρ is a truly random function, its outputs on distinct inputs are independent, uniform 256-bit values. Under $\overline{\text{Coll}}$, the challenge inputs are disjoint from all CPA-phase inputs, so all challenge ρ -outputs are fresh, uniform, and independent of V .

In particular, the keystream masking the varint blob (domain $0\text{x}07$) is:

$$\text{ks}(N^*) = \rho(N^* \| 0\text{x}07 \| \text{be4}(0)) \parallel \rho(N^* \| 0\text{x}07 \| \text{be4}(1)) \parallel \dots \quad (\text{truncated to } |B| \text{ bytes}),$$

where B denotes the varint-encoded token sequence. Each ρ -block is fresh and uniform, and domain $0\text{x}07$ is disjoint from domains $0\text{x}00$ – $0\text{x}06$ used to construct B , so $\text{ks}(N^*)$ is independent of B . Therefore the masked blob satisfies:

$$\tilde{B} = B \oplus \text{ks}(N^*) \stackrel{d}{=} \text{Uniform}(\{0, 1\}^{|B|}),$$

independently of the plaintext bit b and the view V .

Similarly, the authentication tag $T = \rho(N^* \| 0\text{x}03 \| \text{be4}(|aad|) \| aad \| N^* \| \tilde{B})$ is evaluated at an input that has not appeared in any previous ρ evaluation (it is prefixed by N^* and uses domain $0\text{x}03$ with a distinct suffix), so T is a fresh uniform 32-byte value independent of b and V .

Consequently, $c^* = (N^*, \tilde{B}, T)$ is jointly uniformly distributed over its domain, independently of b . The adversary gains no information about b from c^* , so $\Pr[\mathcal{A} \text{ wins } G_1 \mid \overline{\text{Coll}}] = \frac{1}{2}$. $\square$

Combining the bounds.

$$\begin{aligned} \Pr[\mathcal{A} \text{ wins } G_1] &= \Pr[\text{Coll}] \cdot \Pr[\text{wins} \mid \text{Coll}] + \Pr[\overline{\text{Coll}}] \cdot \Pr[\text{wins} \mid \overline{\text{Coll}}] \\ &\leq \Pr[\text{Coll}] + \frac{1}{2} \leq \frac{q^2}{2^{128}} + \frac{1}{2}. \end{aligned}$$

Therefore:

$$\begin{aligned} \text{Adv}_{\text{NAPQES}}^{\text{IND-CPA}}(\mathcal{A}) &= |\Pr[\mathcal{A} \text{ wins } G_0] - \frac{1}{2}| \\ &\leq |\Pr[G_0] - \Pr[G_1]| + |\Pr[\mathcal{A} \text{ wins } G_1] - \frac{1}{2}| \\ &\leq \text{Adv}_F^{\text{PRF}}(\mathcal{B}_1) + \frac{q^2}{2^{128}}. \end{aligned} \quad \square$$

4.2 INT-CTXT: Ciphertext Integrity

Theorem 4.4 (INT-CTXT). *Under the PRF assumption on HMAC-SHA256, NAPQES is INT-CTXT secure. Specifically, for any PPT adversary $\mathcal{A}$ making at most q encryption queries there exists a PRF adversary $\mathcal{B}_2$ with similar running time such that:*

$$\text{Adv}_{\text{NAPQES}}^{\text{INT-CTXT}}(\mathcal{A}) \leq \text{Adv}_{\text{HMAC-SHA256}}^{\text{PRF}}(\mathcal{B}_2) + \frac{q^2}{2^{128}}.$$

Proof. The INT-CTXT experiment requires $\mathcal{A}$ to produce a *fresh* valid ciphertext: a tuple (c^*, aad^*) with $c^* \notin \{c_1, \dots, c_q\}$ (the outputs of the q encryption oracle calls) such that $\text{NAPQES.Dec}(\mathbf{k}, c^*, aad^*) \neq \perp$.

Write $c^* = (N^*, \tilde{B}^*, T^*)$. For Dec to accept, T^* must satisfy:

$$T^* = \text{HMAC}(\text{kb}, 0\text{x}03 \| \text{be4}(|aad^*|) \| aad^* \| N^* \| \tilde{B}^*).$$

Let $x^* = 0\text{x}03 \| \text{be4}(|aad^*|) \| aad^* \| N^* \| \tilde{B}^*$.

Case 1: N^* was not used in any encryption query. All domain-0x03 tag inputs from the query phase are prefixed by nonces $N_1, \dots, N_q \neq N^*$, so x^* was never submitted to $\text{HMAC}(\text{kb}, \cdot)$ during the query phase. Producing the correct T^* is therefore an HMAC MAC forgery at a fresh input. We construct $\mathcal{B}_2$ that simulates the INT-CTXT experiment for $\mathcal{A}$, forwarding all HMAC calls to its own PRF oracle, and uses $\mathcal{A}$'s forgery (x^*, T^*) to distinguish $\text{HMAC}(\text{kb}, \cdot)$ from a random function.

Case 2: $N^* = N_i$ for some query i . Since $c^* \neq c_i = (N_i, \tilde{B}_i, T_i)$, either $\tilde{B}^* \neq \tilde{B}_i$, in which case $x^* \neq x_i$ and the argument from Case 1 applies; or $\tilde{B}^* = \tilde{B}_i$ and $T^* \neq T_i$, but then c^* cannot be a valid ciphertext (the tag input is identical to x_i yet the tag differs from the correct value), so this sub-case cannot contribute to $\mathcal{A}$'s success.

The probability of a nonce collision satisfying $N^* = N_i$ for some i is at most $q/2^{128} \leq q^2/2^{128}$ by the union bound. Combining both cases:

$$\text{Adv}_{\text{NAPQES}}^{\text{INT-CTXT}}(\mathcal{A}) \leq \text{Adv}_{\text{HMAC-SHA256}}^{\text{PRF}}(\mathcal{B}_2) + \frac{q^2}{2^{128}}. \quad \square$$

4.3 AAD Binding

The AAD is included in the tag computation as: $T = \text{HMAC}(\text{kb}, 0\text{x}03\|\text{be}4(|A|)\|A\|N\|B)$. The 4-byte length prefix prevents length-extension mixing between different AAD lengths. Any single-bit change to the AAD produces an unpredictable change to the expected tag, with advantage bounded by the HMAC security.

4.4 IND-CCA Security

Theorem 4.5 (IND-CCA). *Under the PRF assumption on HMAC-SHA256, NAPQES is IND-CCA secure. Specifically, for any PPT adversary $\mathcal{A}$ making at most q encryption queries and q_d decryption queries there exist PRF adversaries $\mathcal{B}_1, \mathcal{B}_2$ with similar running time such that:*

$$\text{Adv}_{\text{NAPQES}}^{\text{IND-CCA}}(\mathcal{A}) \leq \text{Adv}_{\text{HMAC-SHA256}}^{\text{PRF}}(\mathcal{B}_1) + \text{Adv}_{\text{HMAC-SHA256}}^{\text{PRF}}(\mathcal{B}_2) + \frac{(q + q_d)^2}{2^{128}}.$$

Proof. We apply the composition theorem of Bellare and Namprempre [11] (B&N 2000, Theorem 3): a scheme that is simultaneously IND-CPA and INT-CTXT is IND-CCA. We make the argument explicit via two hybrid games.

Game H_0 (Real IND-CCA). The standard IND-CCA experiment: $\mathcal{A}$ has access to an encryption oracle $\mathcal{E}$ and a decryption oracle $\mathcal{D}$. After receiving the challenge ciphertext c^* , $\mathcal{A}$ may call $\mathcal{D}$ on any $c \neq c^*$.

Game H_1 (Decryption oracle stubbed to $\perp$). Identical to H_0 , except $\mathcal{D}$ always returns $\perp$.

Transition $H_0 \rightarrow H_1$. The two games differ only when $\mathcal{A}$ submits a fresh valid ciphertext $c \neq c^*$ to $\mathcal{D}$ that would be accepted in H_0 . Any distinguisher between H_0 and H_1 therefore yields an INT-CTXT forger $\mathcal{A}'$ (with q_d forgery attempts):

$$|\Pr[\mathcal{A} \text{ wins } H_0] - \Pr[\mathcal{A} \text{ wins } H_1]| \leq \text{Adv}_{\text{NAPQES}}^{\text{INT-CTXT}}(\mathcal{A}') \leq \text{Adv}_{\text{HMAC-SHA256}}^{\text{PRF}}(\mathcal{B}_2) + \frac{q_d^2}{2^{128}},$$

where the second inequality applies Theorem 2 with q_d queries.

Game H_1 reduces to IND-CPA. In H_1 the decryption oracle is always $\perp$, providing no information to $\mathcal{A}$. The IND-CCA experiment in H_1 is therefore computationally equivalent to the IND-CPA experiment, giving:

$$\Pr[\mathcal{A} \text{ wins } H_1] \leq \frac{1}{2} + \text{Adv}_{\text{NAPQES}}^{\text{IND-CPA}}(\mathcal{A}'') \leq \frac{1}{2} + \text{Adv}_{\text{HMAC-SHA256}}^{\text{PRF}}(\mathcal{B}_1) + \frac{q^2}{2^{128}},$$

where Theorem 1 is applied to the IND-CPA adversary $\mathcal{A}''$.

Combining.

$$\begin{aligned} \text{Adv}_{\text{NAPQES}}^{\text{IND-CCA}}(\mathcal{A}) &= |\Pr[\mathcal{A} \text{ wins } H_0] - \frac{1}{2}| \\ &\leq |\Pr[H_0] - \Pr[H_1]| + |\Pr[H_1] - \frac{1}{2}| \\ &\leq \text{Adv}_{\text{HMAC-SHA256}}^{\text{PRF}}(\mathcal{B}_2) + \frac{q_d^2}{2^{128}} + \text{Adv}_{\text{HMAC-SHA256}}^{\text{PRF}}(\mathcal{B}_1) + \frac{q^2}{2^{128}}. \end{aligned}$$

Since $(q^2 + q_d^2) \leq (q + q_d)^2$, this gives the stated bound. $\square$

5 Post-Quantum Analysis

SHA-256 and Grover’s algorithm. Grover’s algorithm [4] provides a quadratic speedup for unstructured search. Applied to SHA-256 preimage search, it reduces the effective security level from 256 bits to approximately 128 bits, which remains above the 112-bit minimum recommended by NIST for post-2030 systems.

No Shor-applicable structure. NAPQES does not use discrete logarithms, integer factorisation, or any algebraic group structure. Shor’s algorithm and related quantum algorithms provide no speedup against the construction.

Key space. With a 10-element prime-tuple key from $\mathcal{P}$, the key space is $\approx 2^{196}$. Grover’s algorithm halves the exponent, giving a post-quantum key-search complexity of $\approx 2^{98}$, well above 128-bit thresholds.

6 Comparison with AES-GCM and ChaCha20-Poly1305

Property	NAPQES	AES-GCM	ChaCha20-Poly1305	Ascon
Primitive basis	HMAC-SHA256	AES + GHASH	ChaCha20 + Poly1305	Ascon permutation
FIPS-approved	Yes	Yes	No	No (pending)
Shor-applicable structure	No	No	No	No
Forbidden Attack risk	No	Yes (nonce reuse)	Yes (nonce reuse)	No
Nonce-reuse key recovery	No	Yes (GCM key)	Yes (Poly1305 key)	Partial
Ciphertext expansion	4–20×	1×	1×	1×
Key size	50+ bytes	32 bytes	32 bytes	16 bytes

Table 2: Comparison of NAPQES with established AEAD schemes.

The primary trade-off is ciphertext expansion: NAPQES’s noise-injection mechanism produces 4–20× ciphertext expansion depending on the derived noise probability, compared to no expansion for AES-GCM and ChaCha20-Poly1305. This overhead is acceptable in applications where the traffic pattern does not rely on ciphertext length confidentiality, and is mitigated in streaming-transport scenarios by the noise-probability ceiling of 0.99.

7 Implementation

7.1 Python Reference Implementation

The authoritative reference implementation is `napqes.py`. It is pure Python 3.10+, with no dependencies beyond the standard library (`hmac`, `hashlib`, `secrets`, `base64`). All KAT vectors are derived from this implementation.

7.2 Rust Implementation

A Rust port (`rust/src/lib.rs`) implements the same wire format and passes all negative KAT vectors. Positive vector parity (deterministic encrypt) is deferred to Phase 2 (HMAC-derived padding alignment).

7.3 Constant-Time Considerations

Authentication tag comparison uses `hmac.compare_digest` (Python) and `constant_time_eq` from the `subtle` crate (Rust), providing constant-time equality on the tag bytes. Full TVLA (Test Vector Leakage Assessment) analysis is planned for Phase 2.

8 NIST SP 800-22 Statistical Analysis

We applied the full NIST SP 800-22 Rev 1a Statistical Test Suite to 10^7 bits of NAPQES v6 ciphertext output generated by encrypting a cycling corpus of printable-ASCII plaintext with a 10-element prime key.

All 15 eligible SP 800-22 tests passed at the $\alpha = 0.01$ significance level. Results are reproducible using `tests/sts_pipeline.py` in the reference implementation repository.

[Table of per-test p -values to be inserted from `sts_pipeline.py -out report.json` output prior to publication.]

9 Known-Answer Tests

The KAT corpus (`tests/kat/v6_vectors.json`) contains 30 vectors:

- **V001–V022** (positive): cover empty message, single character, boundary block sizes (1, 7, 15, 16, 31, 32, 128, 256, 1000 characters), 1-element and 10-element keys, non-empty and binary AAD, multiple nonces for the same message, nonce-reuse determinism (V021), and nonce-reuse differentiation (V022).
- **N001–N008** (negative): cover auth tag tamper, too-short ciphertext, wrong key, wrong AAD, legacy unauthenticated blob, nonce flip, single-bit AAD tamper (N007), and v3-format colon-delimited blob presented to the v6 decoder (N008).

Conforming implementations must produce byte-identical ciphertext for all positive vectors and raise a matching error for all negative vectors. The generator (`tests/gen_kats.py`) can be used to verify parity on any port.

10 Fuzz Testing

Two fuzz harnesses are provided:

Python (atheris). `tests/fuzz_atheris.py` targets `decrypt_bytes`, `_b128_decode_tokens`, and the legacy unauthenticated path. It verifies that no exception other than `ValueError` escapes from the public API. During fuzz testing, we discovered that the legacy `_decode_v5_unauth` path propagated `IndexError` on truncated varints; this was fixed by converting to `ValueError` (see `napqes.py`, function `_decode_v5_unauth`).

Rust (cargo-fuzz). `rust/fuzz/fuzz_targets/decode_bytes.rs` targets the Rust `decrypt_bytes` entry point with three key sizes and two AAD values. No panics were observed across 10^6 corpus entries.

11 Known Caveats

The following known caveats are documented in `docs/CAVEATS.md` and `SPEC.md §11` in the reference implementation.

CAV-001 — Streaming RUP (fixed). Earlier versions of `decrypt_stream` yielded plaintext before the authentication tag was verified, allowing an active attacker to cause unverified data to reach the application. This is resolved by the online-AE streaming format (Section 3.8): `decrypt_stream_ae` verifies each chunk’s HMAC-SHA256 tag before releasing any of its plaintext, and a final sentinel tag authenticates the total chunk count. The legacy `decrypt_stream` API is retained with an explicit opt-in gate (`enable_unauthenticated_streaming=True`); new code should use `encrypt_stream_ae` / `decrypt_stream_ae`.

CAV-002 — 16-bit plaintext length cap (open, low). The v6 block padding scheme stores the plaintext length as a 2-byte big-endian integer, capping block-mode messages at 65 535 codepoints. Exceeding the cap raises `ValueError` immediately; there is no silent truncation. Callers requiring larger messages must split at the application layer. A 4-byte length prefix is planned for the v7 wire format.

CAV-003 — Padding length-bucket leakage (open, low). Block-mode ciphertext length reveals the power-of-two padding bucket $\{16, 32, \dots, 65536\}$, leaking up to $\lceil \log_2 n \rceil$ bits of length information. Callers needing full length-hiding must layer a fixed-frame transport. A fixed-frame padding option is planned for the v7 wire format.

CAV-004 — Ciphertext expansion (informational). Noise injection produces 4–20× ciphertext expansion relative to plaintext size. This is a deliberate confidentiality feature, not a bug; it is documented in all datasheets. No fix is planned.

12 Conclusion

We have presented NAPQES, an AEAD scheme whose entire security reduces to the pseudo-randomness of HMAC-SHA256. The construction avoids all algebraic structures targeted by known quantum algorithms, relies solely on FIPS-approved primitives, provides a conventional RFC 5116-compatible interface, and has been validated against a 30-vector KAT corpus, the full NIST SP 800-22 statistical suite, and a fuzz harness covering the complete decoder pipeline.

The streaming Release of Unverified Plaintext issue (CAV-001) has been resolved by the online-AE streaming format, which attaches a per-chunk HMAC-SHA256 tag and a final sentinel tag, giving `decrypt_stream_ae` true online-AE semantics with no plaintext released before verification. Remaining open limitations include ciphertext expansion (inherent in the noise-injection design, CAV-004), a 16-bit block-mode plaintext length cap (CAV-002), and a padding length-bucket leak (CAV-003); these are documented and assigned to future wire-format versions.

Future work includes a TVLA side-channel analysis of the Rust constant-time implementation, a FIPS 140-3 module boundary definition, and a performance evaluation on embedded ARM Cortex-M4 and RISC-V targets.

References

- [1] M. Dworkin, *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC*, NIST SP 800-38D, November 2007.
- [2] Y. Nir, A. Langley, *ChaCha20 and Poly1305 for IETF Protocols*, RFC 7539, May 2015.
- [3] C. Dobraunig, M. Eichlseder, F. Mendel, M. Schl  ffer, *Ascon v1.2: Lightweight Authenticated Encryption and Hashing*, Journal of Cryptology, 34(3), 2021.
- [4] L. K. Grover, *A fast quantum mechanical algorithm for database search*, Proceedings of STOC '96, pp. 212–219, 1996.
- [5] NIST, *Status Report on the Third Round of the NIST Post-Quantum Cryptography Standardization Process*, NIST IR 8413, July 2022.
- [6] A. Joux, *Authentication failures in NIST version of GCM*, NIST Comment, 2006.
- [7] T. Iwata, K. Ohashi, K. Minematsu, *Breaking and Repairing GCM Security Proofs*, CRYPTO 2012, LNCS 7417, pp. 31–49.
- [8] H. Krawczyk, P. Eronen, *HMAC-based Extract-and-Expand Key Derivation Function (HKDF)*, RFC 5869, May 2010.
- [9] E. Barker, J. Kelsey, *Recommendation for Random Number Generation Using Deterministic Random Bit Generators*, NIST SP 800-90A Rev 1, June 2015.
- [10] M. Bellare, R. Canetti, H. Krawczyk, *Keying Hash Functions for Message Authentication*, CRYPTO 1996, LNCS 1109, pp. 1–15.
- [11] M. Bellare, C. Namprempre, *Authenticated Encryption: Relations among Notions and Analysis of the Generic Composition Paradigm*, ASIACRYPT 2000, LNCS 1976, pp. 531–545.
- [12] H. Krawczyk, M. Bellare, R. Canetti, *HMAC: Keyed-Hashing for Message Authentication*, RFC 2104, February 1997.
- [13] A. Rukhin, J. Soto, J. Nechvatal, et al., *A Statistical Test Suite for Random and Pseudo-random Number Generators for Cryptographic Applications*, NIST SP 800-22 Rev 1a, April 2010.